

Министерство науки и высшего образования РФ Пензенский
государственный университет

Кафедра «Системы автоматизированного проектирования»

ОТЧЁТ

по лабораторной работе №2

по дисциплине «Логика и основы алгоритмизации в ИЗ»

на тему «Оценка времени выполнения программ»

Выполнили ст. Гр 25ВВВ4:

Волосунин Е.

Колос М.

Приняли:

Юрова О.В.

Деев М.В.

Пенза 2026

Задание 1.

1. Вычислить порядок сложности программы (O-символику).

```
#include <stdlib.h>
#include <time.h>

int main(void)
{
    setvbuf(stdin, NULL, _IONBF, 0);
    setvbuf(stdout, NULL, _IONBF, 0);

    K clock_t start, end; // объявляем переменные для определения времени выполнения;

    int i=0, j=0, r;
    int a[200][200], b[200][200], c[200][200], elem_c;

    srand(time(NULL)); // инициализируем параметры генератора случайных чисел
    while(i<200)
    {
        J while(j<200)
        {
            G D a[i][j]=rand()% 100 + 1; // заполняем массив случайными числами
            j++;
            i++;
        }
    }
    L srand(time(NULL)); // инициализируем параметры генератора случайных чисел
    i=0; j=0;
    while(i<200)
    {
        I F C b[i][j]=rand()% 100 + 1; // заполняем массив случайными числами
        j++;
        i++;
    }

    H for(i=0; i<200; i++)
    {
        E B A elem_c=0;
        for(r=0; r<200; r++)
        {
            elem_c=elem_c+a[i][r]*b[r][j];
            c[i][j]=elem_c;
        }
    }

    return(0);
}
```

$O(A) = O(1)$; $O(B) = O(1)$; $O(C) = O(N^2)$; $O(D) = O(N)$; $O(E) = O(1)$; $O(F) = O(1)$; $O(G) = O(N^2)$; $O(H) = O(N)$; $O(I) = O(1)$; $O(J) = O(N^3)$; $O(K) = O(N^2)$; $O(L) = O(N)$; $O(L) = \text{sum}(O(A), O(B), O(C), O(F), O(G), O(J)) = O(2N^2 + N^3)$

Итоговая сложность программы $O(2N^2 + N^3)$.

2. Оценить время выполнения программы и кода, выполняющего перемножение матриц, используя функции библиотеки *time.h* для матриц размерами от 100, 200, 400, 1000, 2000, 4000, 10000.

Замер времени выполнения кода после размера матриц = 2000 является нецелесообразным из-за длительности выполнения. Помимо прочего, код в методичке

не работоспособен при значениях больших 300 из-за переполнения стека при инициализации матриц.

Код был отредактирован для работоспособности: инициализация матриц происходит динамически, замер времени работы производится на числах 100, 200, 400, 600, 800, 1000, 1200, 1400, 1600, 1800, 2000. Результат:

```
egorka@raspberrypi:~/Desktop/pgu/2c/log/lab2 $ gcc -o p task1.c
egorka@raspberrypi:~/Desktop/pgu/2c/log/lab2 $ ./p
100 n: 0.007 s
200 n: 0.041 s
400 n: 0.264 s
600 n: 0.986 s
800 n: 2.513 s
1000 n: 9.603 s
1200 n: 18.096 s
1400 n: 28.277 s
1600 n: 48.242 s
1800 n: 73.427 s
2000 n: 94.420 s
```

3. Построить график зависимости времени выполнения программы от размера матриц и сравнить полученный результат с теоретической оценкой.

Рассчитаем теоретическую скорость выполнения программы, вычислив коэффициент по одной полученной точке (не самой маленькой):

$$1000^3 \cdot k = 9,603$$

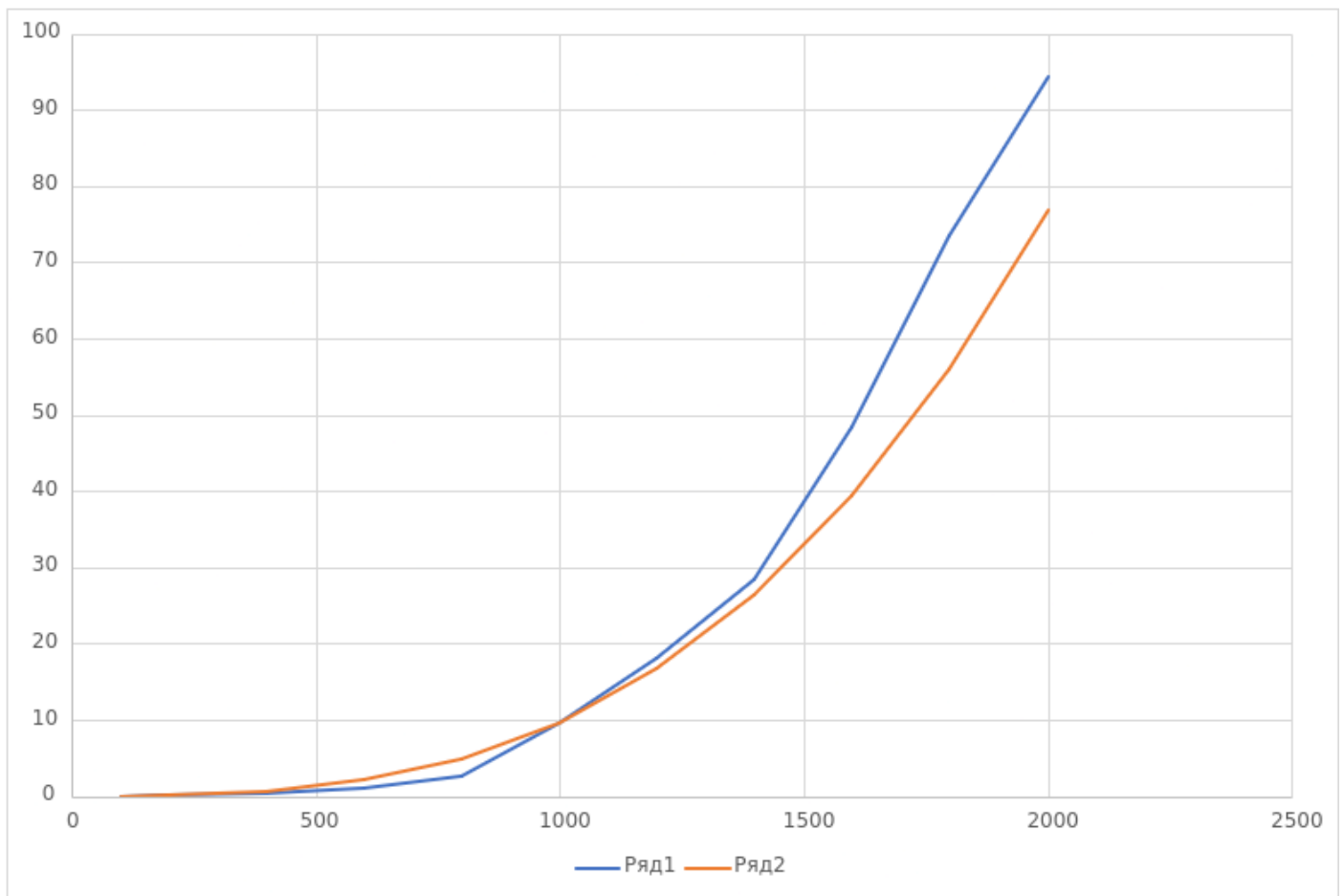
$$k = 9,603 / 1000^3$$

$$k = 0,0000000096$$

Дальше умножим каждое n на коэффициент и получим теоретическое время выполнения $O(2N^2 + N^3)$. Итоговая таблица:

размер	время выполнения	теоретическое время
100	0,007	0,009792
200	0,041	0,077568
400	0,264	0,617472
600	0,986	2,080512
800	2,513	4,927488
1000	9,603	9,6192
1200	18,096	16,616448
1400	28,277	26,380032
1600	48,242	39,370752
1800	73,427	56,049408
2000	94,42	76,8768

График:



Видно, что графики не сходятся, и чем больше размер матриц, тем больше разница. Это вызвано тем, что в коде помимо умножения матриц, происходят и другие процессы: инициализация матриц, их заполнение случайными числами, освобождение выделенной памяти. Все это и влияет на реальное время выполнения.

Псевдокод:

АЛГОРИТМ УмножениеМатриц (N)

ВЫДЕЛИТЬ_ПАМЯТЬ под матрицы A, B, C размером N x N

ИНИЦИАЛИЗИРОВАТЬ_ГЕНЕРАТОР_СЛУЧАЙНЫХ_ЧИСЕЛ()

ДЛЯ i ОТ 0 ДО N-1:

 ДЛЯ j ОТ 0 ДО N-1:

 A[i][j] := СЛУЧАЙНОЕ_ЧИСЛО(1, 100)

 B[i][j] := СЛУЧАЙНОЕ_ЧИСЛО(1, 100)

ДЛЯ i ОТ 0 ДО N-1:

 ДЛЯ j ОТ 0 ДО N-1:

 элемент := 0

 ДЛЯ r ОТ 0 ДО N-1:

 элемент := элемент + A[i][r] * B[r][j]

 C[i][j] := элемент

ОСВОБОДИТЬ_ПАМЯТЬ (A, B, C)

КОНЕЦ АЛГОРИТМА

АЛГОРИТМ Главный

Размеры := [100, 200, 400, 600, 800, 1000, 1200, 1400, 1600, 1800, 2000]

ДЛЯ каждого N ИЗ Размеры:

среднее := 0

ДЛЯ повтор ОТ 1 ДО 3:

t1 := ТЕКУЩЕЕ_ВРЕМЯ()

ВЫЗВАТЬ УмножениеМатриц(N)

t2 := ТЕКУЩЕЕ_ВРЕМЯ()

среднее := среднее + (t2 - t1)

среднее := среднее / 3

ВЫВЕСТИ N, "n:", среднее, "s"

КОНЕЦ АЛГОРИТМА

Задание 2.

1. Оценить время работы каждого из реализованных алгоритмов на случайном наборе значений массива.
2. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой возрастающую последовательность чисел.
3. Оценить время работы каждого из реализованных алгоритмов на массиве, представляющем собой убывающую последовательность чисел.
4. Оценить время работы каждого из реализованных алгоритмов на массиве, одна половина которого представляет собой возрастающую последовательность чисел, а вторая, – убывающую.
5. Оценить время работы стандартной функции *qsort*, реализующей алгоритм быстрой сортировки на выше указанных наборах данных.

Все 5 заданий были реализованы в одной программе. Каждый тип массива генерируется отдельной функцией, после замеряется скорость работы на заданном массиве для каждого из 3-х алгоритмов сортировки - тоже отдельными функциями. Результат выводиться на экран.

```

egorka@raspberrypi:~/Desktop/pgu/2c/log/lab2 $ gcc -o p task2.c
egorka@raspberrypi:~/Desktop/pgu/2c/log/lab2 $ ./p
N = 100000
тип массива:  shell      qs      qsort
случайный:     1.208189,  0.004546,  0.003806
возрастающий:  0.003043,  0.003888,  0.003773
убывающий:     2.280036,  0.003898,  0.003723
воз+убыв      1.227354,  0.004123,  0.003714

```

В результате мы видим стабильность работы алгоритмов qs и qsort, при этом стандартный алгоритм быстрой сортировки qsort работает немного быстрее. Алгоритм shell показывает самый быстрый результат на возрастающем массиве, когда как на остальных скорость работы оставляет желать лучшего.

Псевдокод:

АЛГОРИТМ ШеллСортировка (массив A, количество N)

K := [9, 5, 3, 2, 1] // набор шагов

ДЛЯ каждого шага gap ИЗ K:

ДЛЯ i ОТ gap ДО N-1:

x := A[i]

j := i - gap

ПОКА (j >= 0) И (x < A[j]):

A[j + gap] := A[j]

j := j - gap

A[j + gap] := x

КОНЕЦ АЛГОРИТМА

АЛГОРИТМ БыстраяСортировка (массив A, left, right)

i := left

j := right

x := A[(left + right) / 2] // опорный элемент

ПОВТОРЯТЬ

ПОКА A[i] < x И i < right:

i := i + 1

ПОКА x < A[j] И j > left:

j := j - 1

ЕСЛИ i <= j ТО

обменять местами A[i] и A[j]

i := i + 1

j := j - 1

ДО ТЕХ ПОР, ПОКА i > j

ЕСЛИ left < j ТО БыстраяСортировка(A, left, j)

ЕСЛИ $i < \text{right}$ ТО БыстраяСортировка(A, i, right)

КОНЕЦ АЛГОРИТМА

АЛГОРИТМ ГенерацияСлучайного (массив A, N)

ДЛЯ i ОТ 0 ДО $N-1$:

$A[i] := \text{СЛУЧАЙНОЕ_ЧИСЛО}(1, 1000)$

КОНЕЦ АЛГОРИТМА

АЛГОРИТМ ГенерацияВозрастающего (массив A, N)

$A[0] := \text{СЛУЧАЙНОЕ_ЧИСЛО}(1, 10)$

ДЛЯ i ОТ 1 ДО $N-1$:

$A[i] := A[i-1] + \text{СЛУЧАЙНОЕ_ЧИСЛО}(1, 10)$

КОНЕЦ АЛГОРИТМА

АЛГОРИТМ ГенерацияУбывающего (массив A, N)

$A[0] := \text{СЛУЧАЙНОЕ_ЧИСЛО}(1000001, 10000000)$

ДЛЯ i ОТ 1 ДО $N-1$:

$A[i] := A[i-1] - \text{СЛУЧАЙНОЕ_ЧИСЛО}(1, 10)$

КОНЕЦ АЛГОРИТМА

АЛГОРИТМ ГенерацияСмешанного (массив A, N)

$A[0] := \text{СЛУЧАЙНОЕ_ЧИСЛО}(1, 10)$

ДЛЯ i ОТ 1 ДО $N/2 - 1$:

$A[i] := A[i-1] + \text{СЛУЧАЙНОЕ_ЧИСЛО}(1, 10)$

ДЛЯ i ОТ $N/2$ ДО $N-1$:

$A[i] := A[i-1] - \text{СЛУЧАЙНОЕ_ЧИСЛО}(1, 10)$

КОНЕЦ АЛГОРИТМА

АЛГОРИТМ ИзмеритьВремя (процедура_сортировки, массив A, N)

$t1 := \text{ТЕКУЩЕЕ_ВРЕМЯ}()$

ВЫЗВАТЬ процедура_сортировки(A, N)

$t2 := \text{ТЕКУЩЕЕ_ВРЕМЯ}()$

ВЕРНУТЬ $t2 - t1$

КОНЕЦ АЛГОРИТМА

АЛГОРИТМ Главный

N := 100000

ИНИЦИАЛИЗИРОВАТЬ_ГЕНЕРАТОР_СЛУЧАЙНЫХ_ЧИСЕЛ()

ВЫДЕЛИТЬ_ПАМЯТЬ(массив A, N)

ВЫВЕСТИ "N =", N

ВЫВЕСТИ заголовок таблицы результатов

ДЛЯ каждого типа ИЗ {случайный, возрастающий, убывающий, смешанный}:

 СГЕНЕРИРОВАТЬ_МАССИВ(A, N, тип) // вызвать соответствующий алгоритм генерации

 t_shell := ИзмеритьВремя(ШеллСортировка, A, N)

 t_qs := ИзмеритьВремя(БыстраяСортировка, A, N)

 t_qsort := ИзмеритьВремя(СтандартнаяСортировка, A, N)

 ВЫВЕСТИ тип, t_shell, t_qs, t_qsort

ОСВОБОДИТЬ_ПАМЯТЬ(A)

КОНЕЦ АЛГОРИТМА

Вывод: все задания выполнены в лучшем виде.